Conv2d#

https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html#torch.nn.Conv

Description:

Applies a 2D convolution over an input signal composed of several input planes. In the simplest case, the output value of the layer with input size (N,Cin,H,W)(N, C_{\text{in}}, H, W)(N,Cin,H,W) and output (N,Cout,Hout,Wout)(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}}) (N,Cout,Hout,Wout) can be precisely described as: where ⋆\star⋆ is the valid 2D cross-correlation operator, NNN is a batch size, CCC denotes a number of channels, HHH is a height of input planes in pixels, and WWW is width in pixels. This module supports TensorFloat32. On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

Function Signature

```
class torch.nn.Conv2d(in_channels, out_channels,
kernel_size, stride=1, padding=0, dilation=1, groups=1,
bias=True, padding_mode='zeros', device=None, dtype=None)
[source]#

Parameters

Shape:

Variables
```

Code Examples

```
>>> # With square kernels and equal stride
>>> m = nn.Conv2d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2))
>>> # non-square kernels and unequal stride and with padding and
dilation
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2),
dilation=(3, 1))
>>> input = torch.randn(20, 16, 50, 100)
>>> output = m(input)
```

Notes & Warnings

NOTE Note When groups == in_channels and out_channels == K * in_channels, where K is a positive integer, this operation is also known as a “depthwise convolution”. In other words, for an input of size (N,Cin,Lin)(N, C_{in}, L_{in})(N,Cin,Lin), a depthwise convolution with a depthwise multiplier K can be performed with the arguments (Cin=Cin,Cout=Cin×K,...,groups=Cin)(C_\text{in}=C_\text{in}, C_\text{out}=C_\text{in} \times \text{K}, ..., \text{groups}=C_\text{in})(Cin=Cin,Cout=Cin×K,...,groups=Cin).

NOTE Note In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting torch.backends.cudnn.deterministic = True. See Reproducibility for more information.

NOTE Note padding='valid' is the same as no padding. padding='same' pads the input so the output has the shape as the input. However, this mode doesn’t support any stride values other than 1.

NOTE Note This module supports complex data types i.e. complex32, complex64, complex128.

Detailed Documentation

Documentation

Applies a 2D convolution over an input signal composed of several input planes.

In the simplest case, the output value of the layer with input size (N,Cin,H,W)(N, C_{\text{in}}, H, W)(N,Cin,H,W) and output (N,Cout,Hout,Wout)(N, C_{\text{out}}, H_{\text{out}}, W_{\text{out}})(N,Cout,Hout,Wout) can be precisely described as:

where ⋆\star⋆ is the valid 2D cross-correlation operator, NNN is a batch size, CCC denotes a number of channels, HHH is a height of input planes in pixels, and WWW is width in pixels.

This module supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

stride controls the stride for the cross-correlation, a single number or a tuple.

padding controls the amount of padding applied to the input. It can be either a string {‘valid’, ‘same’} or an int / a tuple of ints giving the amount of implicit padding applied on both sides.

dilation controls the spacing between the kernel points; also known as the à trous algorithm. It is harder to describe, but this link has a nice visualization of what dilation does.

groups controls the connections between inputs and outputs. in_channels and out_channels must both be divisible by groups. For example,

At groups=1, all inputs are convolved to all outputs.

At groups=2, the operation becomes equivalent to having two conv layers side by side, each seeing half the input channels and producing half the output channels, and both subsequently concatenated.

At groups= in_channels, each input channel is convolved with its own set of filters (of size out_channelsin_channels\frac{\text{out_channels}}{\text{in_channels}}in_channelsout_channels).

The parameters kernel_size, stride, padding, dilation can either be:

a single int – in which case the same value is used for the height and width dimension

a tuple of two ints – in which case, the first int is used for the height dimension, and the second int for the width dimension

When groups == in_channels and out_channels == K * in_channels, where K is a positive integer, this operation is also known as a “depthwise convolution”.

In other words, for an input of size (N,Cin,Lin)(N, C_{in}, L_{in})(N,Cin,Lin), a depthwise convolution with a depthwise multiplier K can be performed with the arguments (Cin=Cin,Cout=Cin×K,...,groups=Cin)(C_\text{in}=C_\text{in}, C_\text{out}=C_\text{in} \times \text{K}, ..., \text{groups}=C_\text{in})(Cin=Cin,Cout=Cin×K,...,groups=Cin).

In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting torch.backends.cudnn.deterministic = True. See Reproducibility for more information.

padding='valid' is the same as no padding. padding='same' pads the input so the

output has the shape as the input. However, this mode doesn’t support any stride values other than 1.

This module supports complex data types i.e. complex32, complex64, complex128.

in_channels (int) – Number of channels in the input image

out_channels (int) – Number of channels produced by the convolution

kernel_size (int or tuple) – Size of the convolving kernel

stride (int or tuple, optional) – Stride of the convolution. Default: 1

padding (int, tuple or str, optional) – Padding added to all four sides of the input. Default: 0

dilation (int or tuple, optional) – Spacing between kernel elements. Default: 1

groups (int, optional) – Number of blocked connections from input channels to output channels. Default: 1

bias (bool, optional) – If True, adds a learnable bias to the output. Default: True

padding_mode (str, optional) – 'zeros', 'reflect', 'replicate' or 'circular'. Default: 'zeros'

Input: (N,Cin,Hin,Win)(N, C_{in}, H_{in}, W_{in})(N,Cin,Hin,Win) or (Cin,Hin,Win)(C_{in}, H_{in}, W_{in})(Cin,Hin,Win)

Output: (N,Cout,Hout,Wout)(N, C_{out}, H_{out}, W_{out})(N,Cout,Hout,Wout) or (Cout,Hout,Wout)(C_{out}, H_{out}, W_{out})(Cout,Hout,Wout), where

weight (Tensor) – the learnable weights of the module of shape (out_channels,in_channelsgroups,(\text{out_channels}, \frac{\text{in_channels}}

{\text{groups}},(out_channels,groupsin_channels,
kernel_size[0],kernel_size[1])\text{kernel_size[0]},
\text{kernel_size[1]})kernel_size[0],kernel_size[1]). The values of these weights are sampled from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=groupsCin∗∏i=01kernel_size[i]k = \frac{groups}{C_\text{in} * \prod_{i=0}^{1}\text{kernel_size}[i]}k=Cin∗∏i=01kernel_size[i]groups

bias (Tensor) – the learnable bias of the module of shape (out_channels). If bias is True, then the values of these weights are sampled from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=groupsCin∗∏i=01kernel_size[i]k = \frac{groups}{C_\text{in} * \prod_{i=0}^{1}\text{kernel_size}[i]}k=Cin∗∏i=01kernel_size[i]groups